

Automação e Validação de Interface com Selenium e Pytest

Objetivo: Aplicar os conceitos fundamentais do ecossistema Selenium, criando um script de testes automatizados ponta a ponta (E2E) em uma página HTML local, utilizando boas práticas de seleção de Locators e validações (assertions) com Pytest.

Preparação do Ambiente e Arquivo Alvo

1. Configuração do Projeto:

- Crie uma nova pasta para o seu projeto chamada `automacao_selenium_lab`.
- Crie um ambiente virtual Python e ative-o.
- Instale as bibliotecas necessárias utilizando o terminal.

2. Criação da Página Alvo:

- Dentro da pasta do projeto, crie um arquivo chamado `portal.html` e cole o código abaixo. Esta será a página que você irá automatizar.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Portal do Colaborador</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    .btn-avancado { padding: 10px; margin: 10px 0; border: 1px
solid black; cursor: pointer; width: 250px; text-align: center; }
    .mensagem { color: green; font-weight: bold; display:
none; }
  </style>
</head>
<body>
  <h1>Bem-vindo ao Portal</h1>

  <div class="formulario">
    <label for="nome_usuario">Nome Completo:</label><br>
    <input type="text" id="nome_usuario"
name="usuario_input"><br><br>

    <label for="email">E-mail corporativo:</label><br>
    <input type="email" id="email" name="email_input"><br><br>

    <button id="btn-enviar"
onclick="document.getElementById('msg-
```

```

sucesso').style.display='block';">Enviar Dados</button>
    <p id="msg-sucesso" class="mensagem">Dados enviados com
sucesso!</p>
</div>

<hr>

<h3>Ações Avançadas</h3>
<div id="btn-duplo" class="btn-avancado"
ondblclick="this.innerText='Autorizado!';
this.style.backgroundColor='lightgreen';">
    Duplo clique para Autorizar
</div>

<div id="btn-direito" class="btn-avancado"
oncontextmenu="this.innerText='Menu Aberto!';
this.style.backgroundColor='yellow'; return false;">
    Clique direito para Opções
</div>

<textarea id="obs" rows="4" cols="50">Apague este texto e
insira suas observações.</textarea>

</body>
</html>

```

Construção da Suíte de Testes com Pytest

Agora você vai codificar. Crie um arquivo Python chamado `test_portal.py`. Este arquivo deverá conter a estrutura do Pytest para realizar as validações.

Requisitos do Script:

1. Fixture de Setup e Teardown:

- Crie uma `@pytest.fixture` chamada `navegador`.
- Ela deve inicializar o `ChromeDriver`.
- Ela deve carregar o arquivo `portal.html` local usando o caminho absoluto.
- Ela deve garantir que o navegador seja fechado ao final de cada teste, mesmo se o teste falhar.

2. Cenário de Teste 1: Validação de Título e Formulário Simples

- Crie uma função `test_preencher_formulario(navegador)`.

- Valide usando `assert` se o título da página é "Portal do Colaborador".
- Preencha os campos de "Nome Completo" e "E-mail corporativo".
- Clique no botão "Enviar Dados".
- Valide com `assert` se a mensagem "Dados enviados com sucesso!" está visível na tela (dica: verifique a propriedade `.is_displayed()` do elemento ou confira se o texto dele é correspondente).

3. Cenário de Teste 2: Ações Avançadas de Mouse

- Crie uma função `test_acoes_avancadas_mouse(navegador)`.
- Instancie a classe `ActionChains`.
- Execute um duplo clique no botão "Duplo clique para Autorizar" e use `assert` para verificar se o texto do botão mudou para "Autorizado!".
- Execute um clique com o botão direito no botão "Clique direito para Opções" e verifique com `assert` se o texto mudou para "Menu Aberto!".

4. Cenário de Teste 3: Ações Avançadas de Teclado

- Crie uma função `test_acoes_teclado(navegador)`.
- Localize o campo de texto (`textarea`).
- Usando a classe `Keys` e `ActionChains` (ou o método de atalho do próprio OS), simule as teclas para selecionar todo o texto (`CTRL + A` ou `COMMAND + A`) e apague-o (`BACKSPACE`).
- Envie um novo texto: "Teste automatizado finalizado.".
- Valide se o valor atual dentro do `textarea` (`elemento.get_attribute("value")`) corresponde ao texto que você acabou de digitar.

Entrega do Exercício

Para concluir o exercício, abra seu terminal na pasta do projeto e execute o comando:

```
pytest test_portal.py -v
```

Você deve obter sucesso (PASS) em todos os três cenários criados. Se algum falhar, leia a mensagem de erro do Pytest, corrija seu código e execute novamente. Entregue os arquivos `portal.html` e `test_portal.py`.